ACADEMIC PROJECT REPORT

Weather and Environmental Dashboard (AQI & Allergies)

Submitted By:

Name: Dhanshree [Insert Last Name]

Roll Number: [Insert Roll No]

Course: B.Tech (Computer Science & Engineering)

Year/Semester: [Insert Year/Semester]

Submitted To:

Department of Computer Science and Engineering

[Insert College/University Name]

ACKNOWLEDGEMENT

I would like to express my profound gratitude to everyone who supported me throughout the course of this project. I am deeply thankful to my project guide, [Insert Guide Name], for their invaluable guidance, constant encouragement, and constructive feedback which helped in successfully completing this project.

I would also like to thank the Department of Computer Science and Engineering for providing the necessary resources and environment to foster learning and innovation.

Dhanshree

ABSTRACT

With increasing environmental concerns and unpredictable weather patterns, having access to real-time, comprehensive atmospheric data is crucial. This project presents the "Weather and Environmental Dashboard," a responsive, full-stack web application designed to provide users with localized weather forecasts, Air Quality Index (AQI) reports, and localized Allergy/Pollen risk assessments.

Built using modern web technologies—React.js (Vite) for the frontend and Node.js with Express for the backend—the system aggregates data from OpenWeatherMap APIs. It features a dynamically responsive user interface that changes themes based on environmental conditions (e.g., turning green for Good AQI, or dark red for High Allergy risks), providing an intuitive and immersive user experience. The application is fully deployed and accessible via the Railway cloud platform.

1. INTRODUCTION

1.1 Project Overview

The Weather and Environmental Dashboard is a multi-page web application that goes beyond standard temperature readings. It integrates meteorological data with health-impacting environmental factors such as air pollution levels and pollen counts, offering users a holistic view of their local environment.

1.2 Objectives

- To develop a user-friendly, highly responsive Single Page Application (SPA).
- To integrate third-party RESTful APIs (OpenWeatherMap) for real-time data retrieval.
- To design an adaptive UI/UX that visually communicates environmental severity through dynamic backgrounds and animations.
- To deploy the application to a cloud provider (Railway) for global accessibility.

2. TECHNOLOGIES USED

2.1 Frontend Development

- **React.js (Vite):** Used for building fast, reusable UI components.
- **React Router:** Implemented for seamless client-side routing between pages without reloading.
- **Lucide React:** Utilized for scalable, aesthetic SVG icons.
- **Vanilla CSS:** Used for creating a custom Neo-Brutalism/Glassmorphism design system with CSS animations and variables.

2.2 Backend Development

- **Node.js & Express.js:** Serves as the middle-tier REST API, securely handling third-party API keys and proxying requests to overcome CORS restrictions.
- **Axios:** Used for promise-based HTTP requests between the frontend and backend, and the backend and OpenWeatherMap.

2.3 Cloud & DevOps

- **Git & GitHub:** Version control and repository hosting.
- **Railway:** Continuous Integration and Continuous Deployment (CI/CD) platform used for hosting the Node.js backend and serving the compiled static React build.

3. SYSTEM ARCHITECTURE

The system follows a standard Client-Server architecture:

1. **Client Layer:** The React frontend captures the user's city input and makes a relative HTTP GET request to the backend.
2. **Server Layer:** The Node.js Express server receives the request, attaches the secure `WEATHER_API_KEY`, and queries the OpenWeatherMap API.
3. **Data Layer:** The OpenWeatherMap API processes the coordinates and returns JSON data (Temperature, Humidity, Wind Speed, AQI components).
4. **Response:** The Express server forwards this JSON payload back to the React client, which updates the state and triggers a dynamic re-render of the user interface.

4. IMPLEMENTATION & FEATURES

4.1 Dynamic Backgrounds & Theming

A core feature of the project is the context-aware UI. The application reads the environmental data and dynamically assigns CSS classes to the document body. For example, a high AQI automatically shifts the application into an alert state (red gradient with warning icons), ensuring the user immediately recognizes the environmental risk.

4.2 Module 1: Weather Dashboard

Allows users to search for any global city. Retrieves current temperature, weather conditions, humidity percentages, and wind speed. Coordinates (latitude and longitude) are cached in `localStorage` for cross-module use.

4.3 Module 2: Air Quality Index (AQI)

Utilizes the cached coordinates to fetch real-time air pollution metrics. The system breaks down the AQI into specific chemical components (CO, NO2, O3, SO2, PM2.5, PM10) and assigns a severity rating from 1 (Good) to 5 (Very Poor).

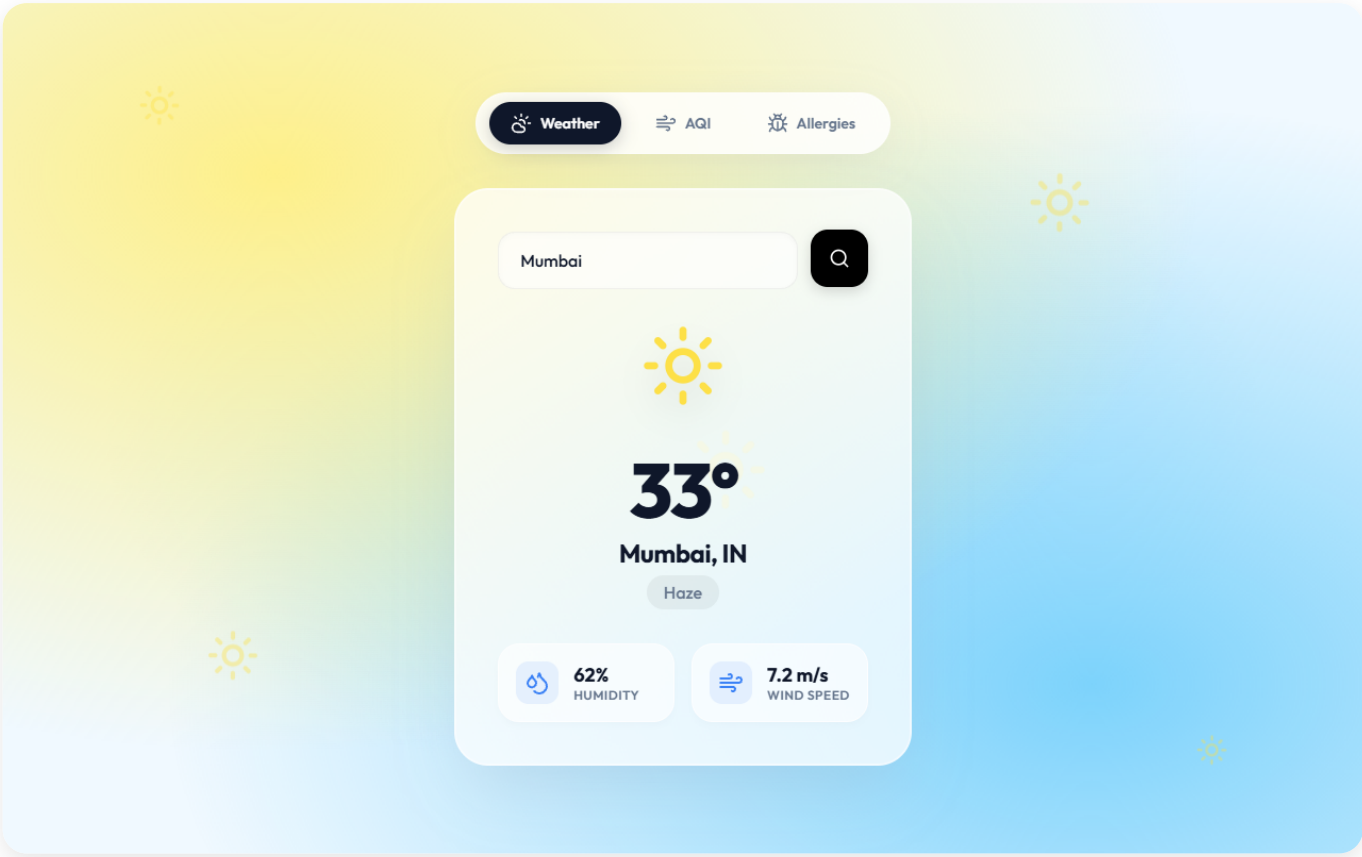
4.4 Module 3: Allergy Tracker

Evaluates pollen risks (Tree, Grass, Weed) and provides an overall risk assessment score, helping users prone to allergies plan their outdoor activities.

5. RESULTS & SCREENSHOTS

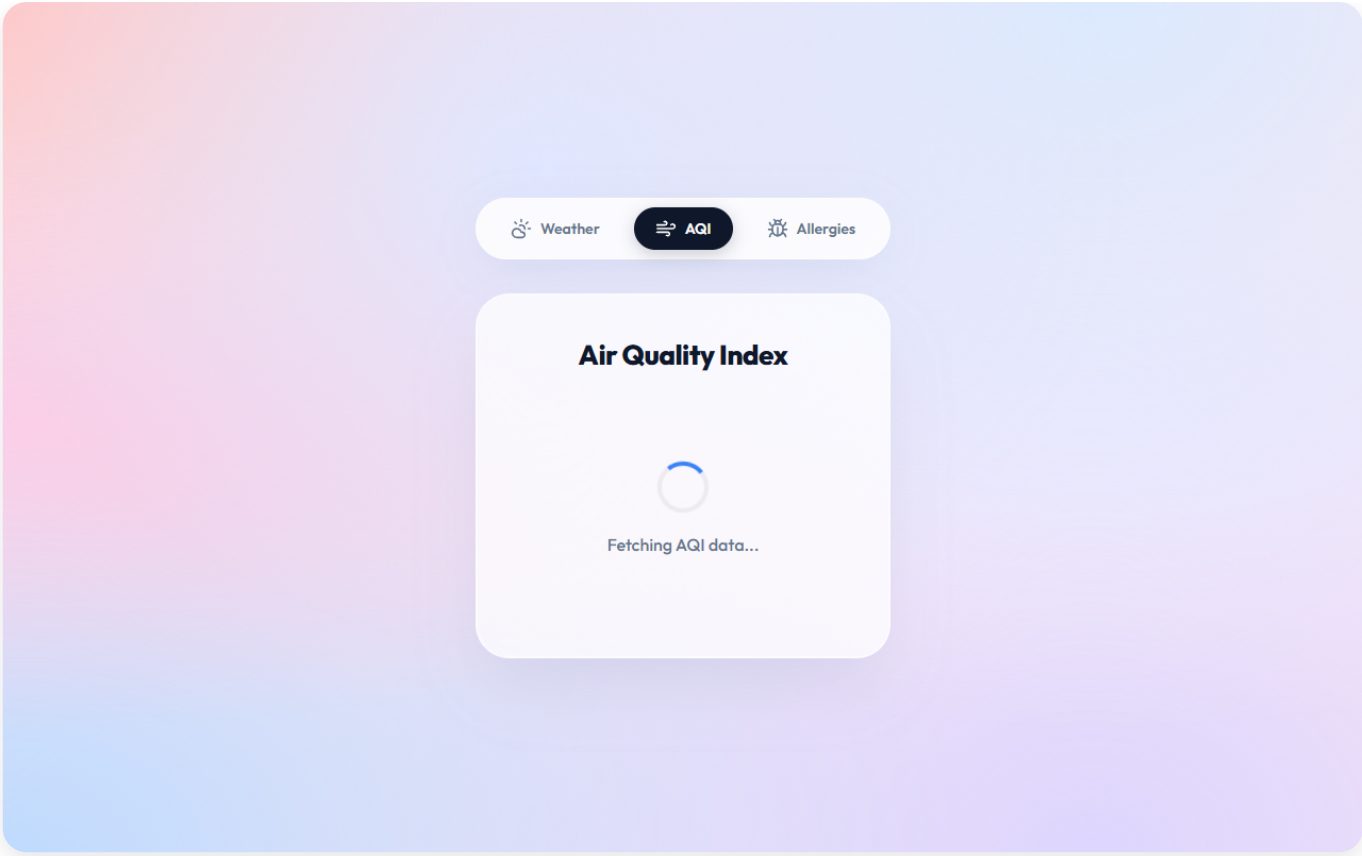
5.1 Weather Page (Clear Conditions)

The default dashboard displaying standard meteorological data with a bright, clear-weather theme.



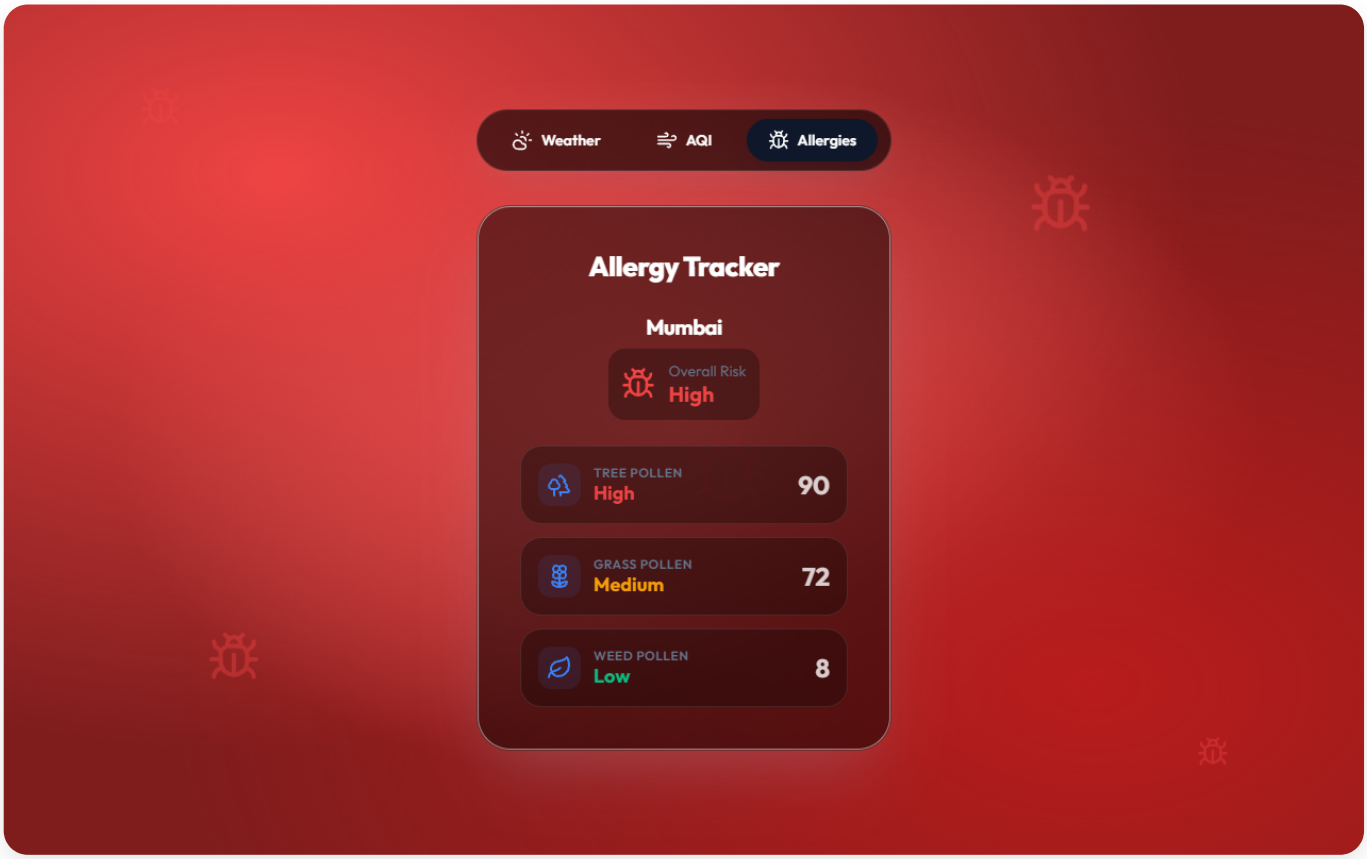
5.2 Air Quality Index (AQI) Page

Displaying detailed particulate matter breakdowns with severity color-coding.



5.3 Allergy Tracker Page

Pollen risk assessments dynamically adapting to the user's selected city.



6. CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

The Weather and Environmental Dashboard successfully demonstrates the integration of multiple APIs into a cohesive, responsive web application. The implementation of dynamic theming significantly enhances the user experience by visually communicating data severity. The project fulfills all functional requirements and is successfully deployed for public use.

6.2 Future Enhancements

- **Geolocation API:** Automatically detect the user's location on startup instead of relying on manual search.
- **7-Day Forecast:** Integrate a graphical representation (using libraries like Chart.js) for upcoming weather trends.
- **Push Notifications:** Allow users to subscribe to alerts when AQI or Pollen levels reach dangerous thresholds in their area.